Practica Render

En esta práctica vamos a construir y publicar en internet una **API REST** que permite guardar y consultar información de estudiantes desde una base de datos. Al finalizar, tendrás una dirección web (URL) pública que cualquier persona podrá usar para consultar tus datos, tal como funcionan las aplicaciones reales que usamos a diario.

¿Qué es una API?

Una **API** (Application Programming Interface / Interfaz de Programación de Aplicaciones) es un conjunto de reglas que permite que dos programas se comuniquen entre sí. Por ejemplo, cuando una app del clima te muestra la temperatura de tu ciudad, en realidad esa app le está "preguntando" a una API de un servicio meteorológico, y la API le responde con los datos. En nuestro caso, la API que construiremos permitirá que cualquier aplicación (una página web, un celular, otro programa) pueda pedirle a nuestra base de datos información de los estudiantes, sin tener que acceder directamente a la base de datos.

¿Qué significa desplegar en la nube?

Cuando programamos en nuestro computador, la API solo funciona mientras nuestro computador esté encendido y ejecutando el programa (esto se llama trabajar en **local** o *localhost*). "Desplegar" (o hacer *deploy*) significa mudar ese programa a un servidor de internet que está encendido 24/7, para que quede disponible para cualquier persona, desde cualquier lugar, sin depender de que tu computador esté prendido. Render es una de varias plataformas que ofrecen este servicio (otras conocidas son Railway, Heroku o AWS).

Antes de programar la API necesitamos un lugar donde guardar la información: una **base de datos**. Usaremos **PostgreSQL**, uno de los motores de bases de datos relacionales más usados en el mundo, alojado en **Neon**, una plataforma que ofrece bases de datos PostgreSQL gratuitas en la nube (esto se conoce como *Database as a Service* o *serverless database*). La ventaja de usar Neon en lugar de instalar PostgreSQL en tu propio computador es que la base de datos queda accesible desde internet, lista para que Render se conecte a ella cuando la API esté desplegada.

Crear la base de datos

Se crea la base de datos en Neon o alguna plataforma en la nube

Tabla estudiante

```
CREATE TABLE estudiantes (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    grado INTEGER NOT NULL,
    promedio NUMERIC(3,2) NOT NULL
);
```

La instrucción anterior está escrita en **SQL** (*Structured Query Language*), el lenguaje que se usa para crear y consultar bases de datos relacionales. Analicemos cada parte:

`CREATE TABLE estudiantes`**: crea una tabla llamada `estudiantes`, que funcionará como una especie de hoja de cálculo dentro de la base de datos.

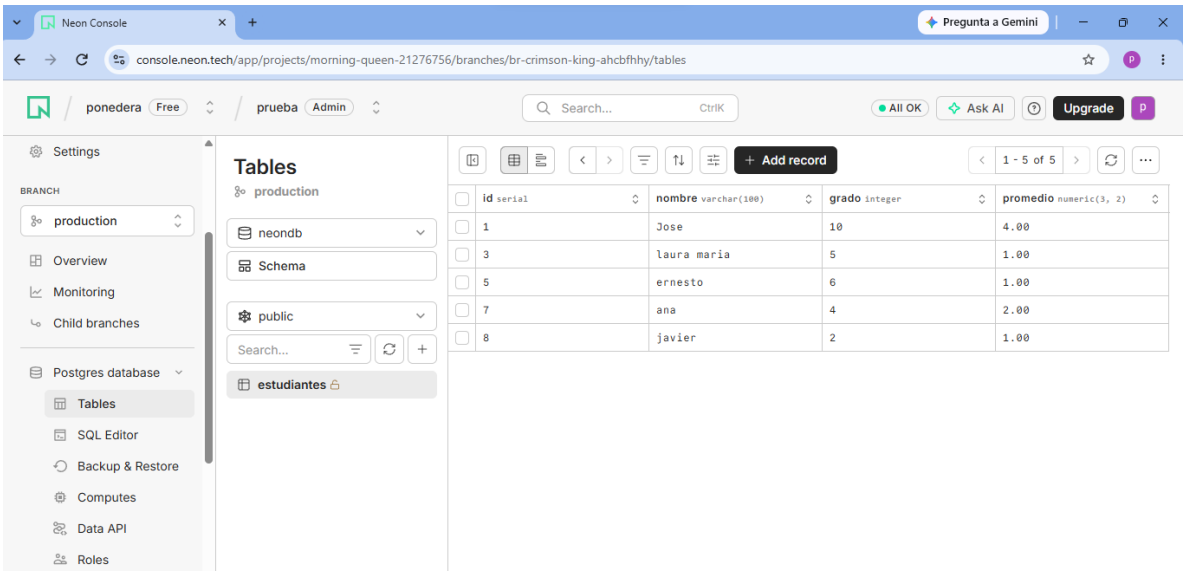
`id SERIAL PRIMARY KEY`**: crea una columna `id` que se llena automáticamente con un número consecutivo (1, 2, 3…) cada vez que se agrega un estudiante. `PRIMARY KEY` (llave primaria) significa que ese valor es único y sirve para identificar sin ambigüedad cada fila de la tabla.

`nombre VARCHAR(100) NOT NULL`**: columna de texto que admite hasta 100 caracteres. `NOT NULL` obliga a que este campo siempre tenga un valor; no se puede dejar vacío.

`grado INTEGER NOT NULL`**: columna numérica entera (sin decimales), también obligatoria.

`promedio NUMERIC(3,2) NOT NULL`**: columna numérica con decimales. `NUMERIC(3,2)` significa que el número puede tener hasta 3 dígitos en total, de los cuales 2 son decimales (por ejemplo, 4.50).

Este diseño de tabla se conoce como el **esquema** de la base de datos, y debe pensarse antes de programar la API, porque la API va a leer y escribir exactamente sobre estas columnas.



Creación de la api

Con la base de datos lista, construimos la API en **Python** usando el framework **FastAPI**, que permite crear rápidamente rutas (llamadas *endpoints*) que responden a peticiones HTTP. Las peticiones más comunes son:

GET: para consultar/leer información (por ejemplo, obtener la lista de estudiantes).

POST: para crear un nuevo registro (por ejemplo, agregar un estudiante).

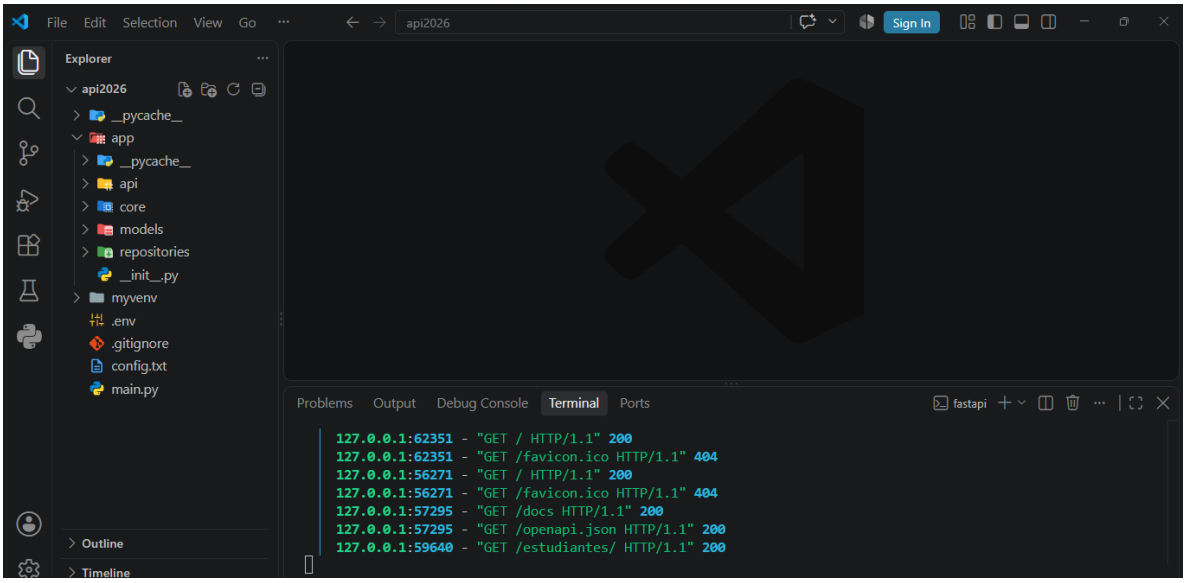
PUT: para actualizar un registro existente.

DELETE: para eliminar un registro.

La API se conecta a la base de datos de Neon usando la librería **psycopg2**, que permite que el código Python envíe instrucciones SQL (como el `CREATE TABLE` que vimos antes) y reciba resultados desde PostgreSQL. Es importante partir de un ejemplo ya construido en clase para no empezar desde cero, e ir modificando los nombres de tablas y columnas según el proyecto de cada estudiante.

Se debe tomar como ejemplo, alguna api realizada en clases.





requirements.txt

Un archivo requirements.txt es un archivo de texto plano que lista todas las librerías externas (dependencias de Python) necesarias para que un proyecto se ejecute correctamente.

Permite registrar exactamente qué paquetes y versiones utiliza tu aplicación para garantizar que funcione de la misma manera en cualquier computadora o entorno de servidor.

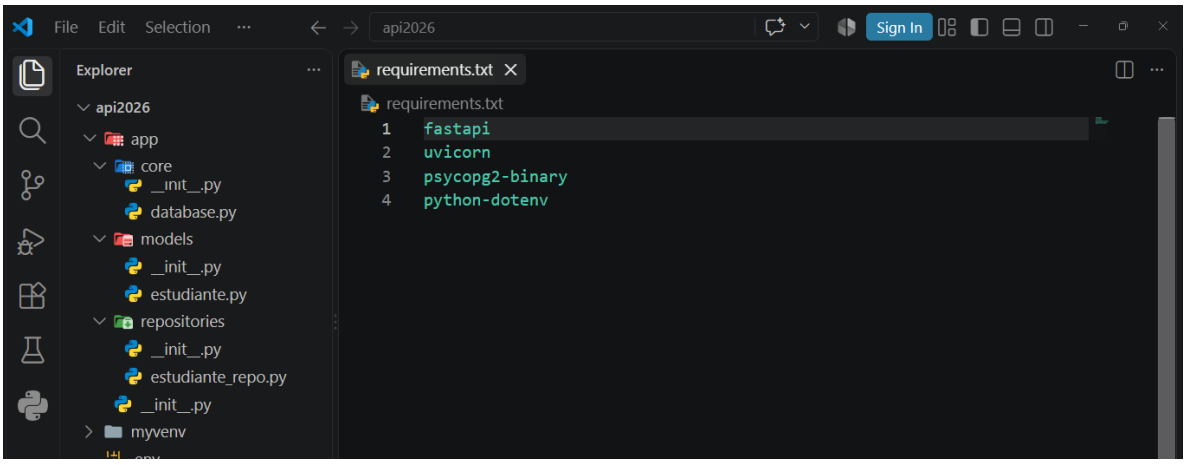
Este archivo es fundamental para el despliegue: cuando subamos el proyecto a Render, la plataforma leerá automáticamente el `requirements.txt` y ejecutará `pip install -r requirements.txt` para instalar cada una de las librerías con la versión indicada, exactamente igual a como funcionaban en el computador donde se desarrolló la API. Si este archivo falta o tiene errores, el despliegue fallará. Los tres paquetes que vemos a continuación son los mínimos necesarios para esta práctica:

fastapi: el framework que define y ejecuta los endpoints de la API.

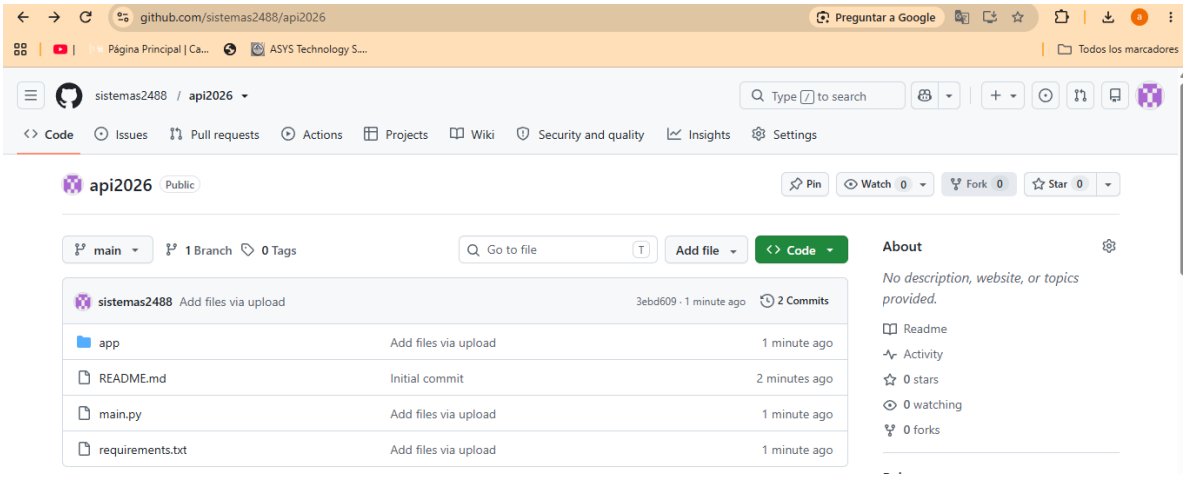
uvicorn: el servidor que realmente "enciende" y ejecuta la aplicación FastAPI.

psycopg2-binary: el conector que permite que Python se comunique con PostgreSQL.

python-dotenv: sirve para cargar variables de entorno desde un archivo .env a la aplicación de Python, permitiendo separar la configuración del entorno de desarrollo del código fuente.



Subir el proyecto a Git

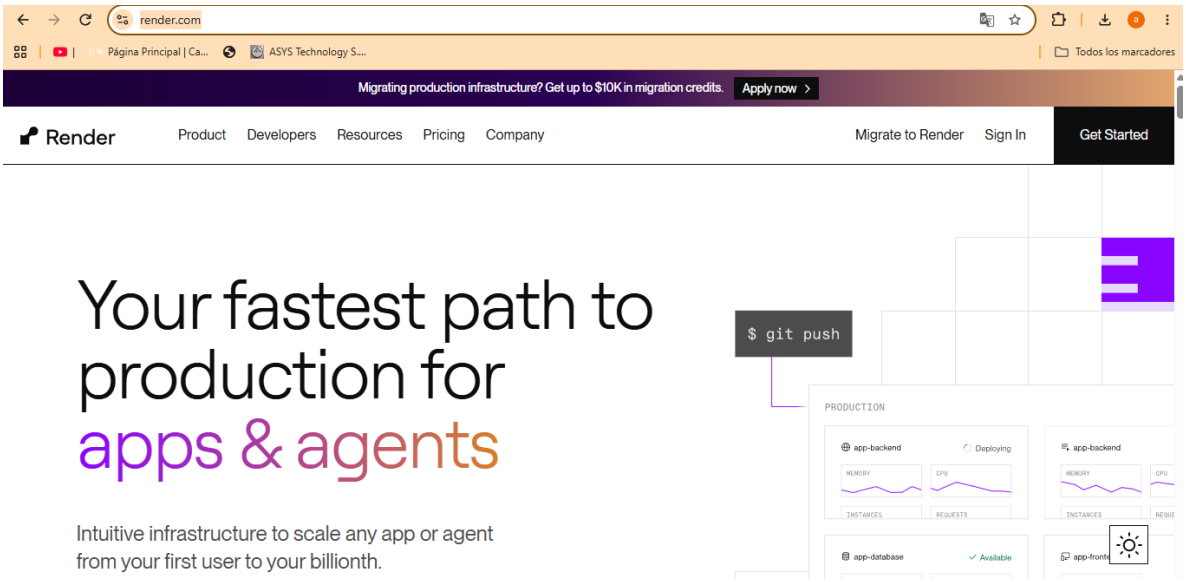


https://github.com/sistemas2488/api2026

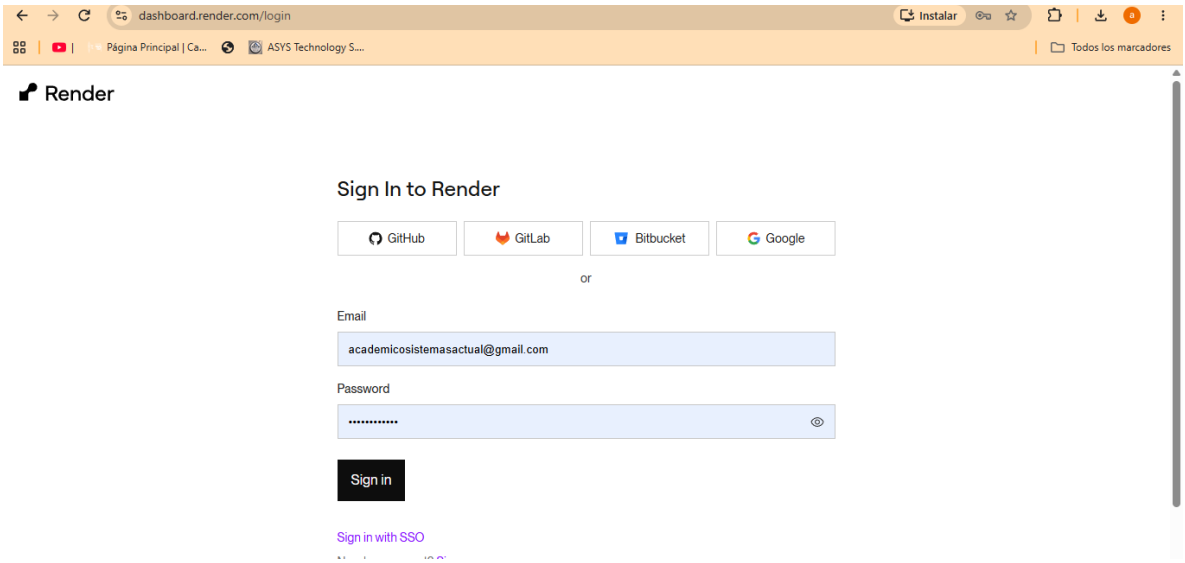
Ingresar a render

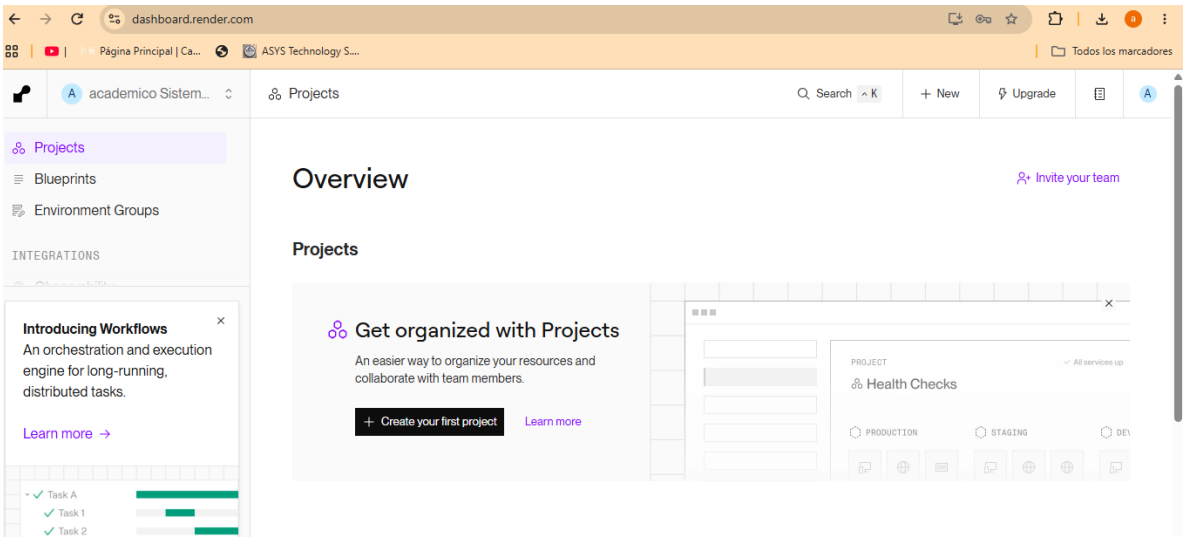
Render es una plataforma de tipo **PaaS** (*Platform as a Service*), que se encarga de todo lo relacionado con servidores, sistema operativo y configuración de red, para que el estudiante solo se preocupe por su código. Al usar el plan gratuito, es importante tener en cuenta que el servicio "se duerme" después de un periodo de inactividad, por lo que la primera petición después de un tiempo sin uso puede tardar unos segundos más en responder (esto se conoce como *cold start* o arranque en frío); no es un error de tu API

https://render.com/



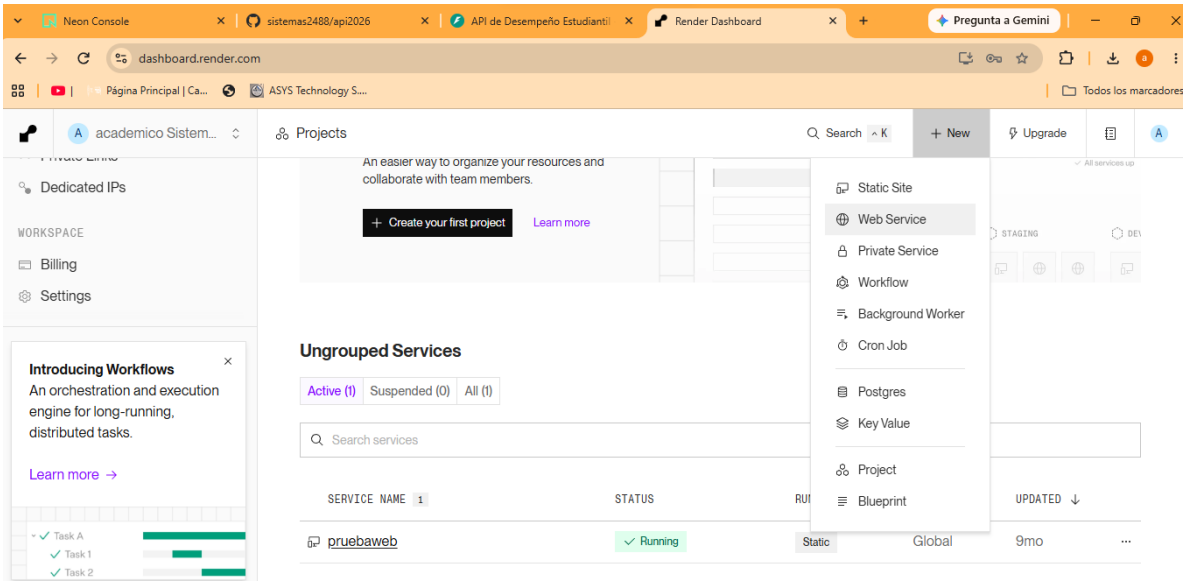
Ingresar las credenciales para acceder mediante GitHub



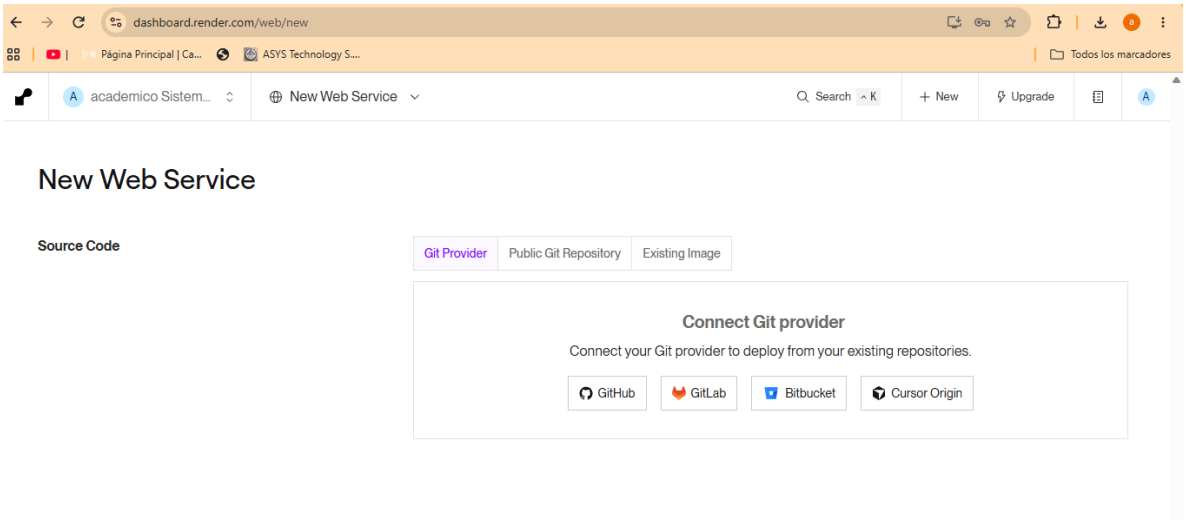


Render ofrece varios tipos de servicio (Web Service, Static Site, Background Worker, entre otros). Para esta práctica seleccionamos **Web Service**, que es el tipo pensado para aplicaciones que responden a peticiones HTTP, como nuestra API construida con FastAPI

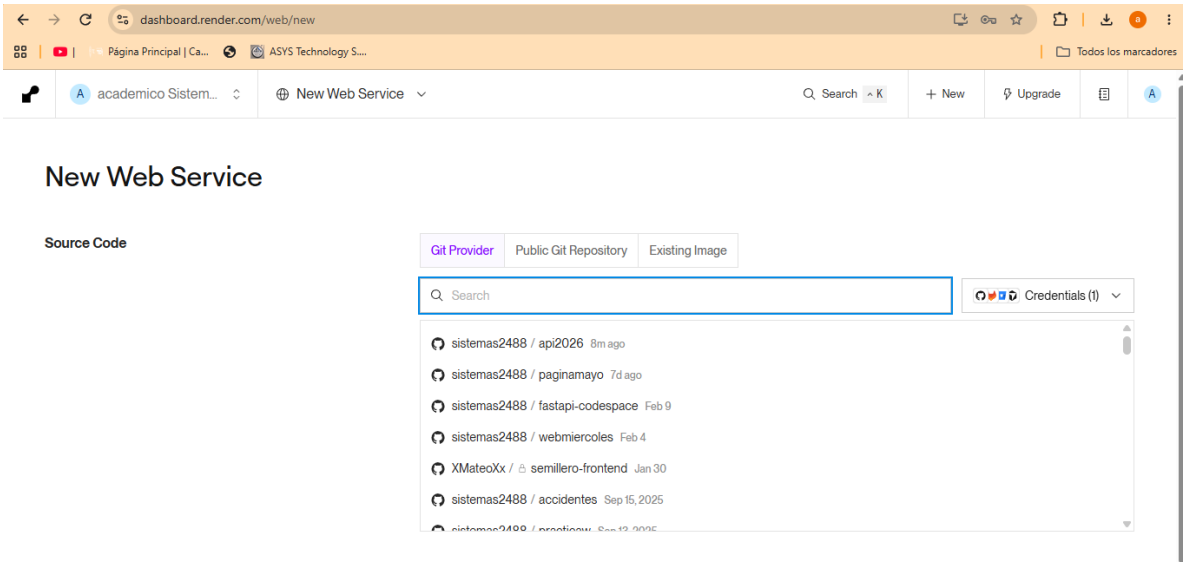
New → WebService



Conectar con Git



Buscar el repositorio



Nombre: api_estudiante

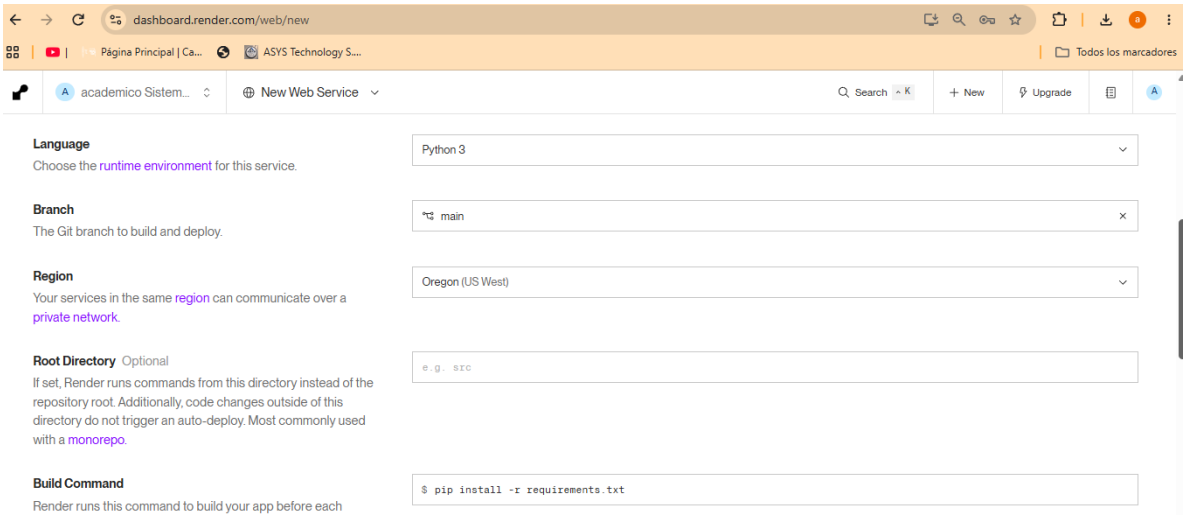
En esta parte del formulario, además del nombre del servicio, Render pide otros datos clave para el despliegue:

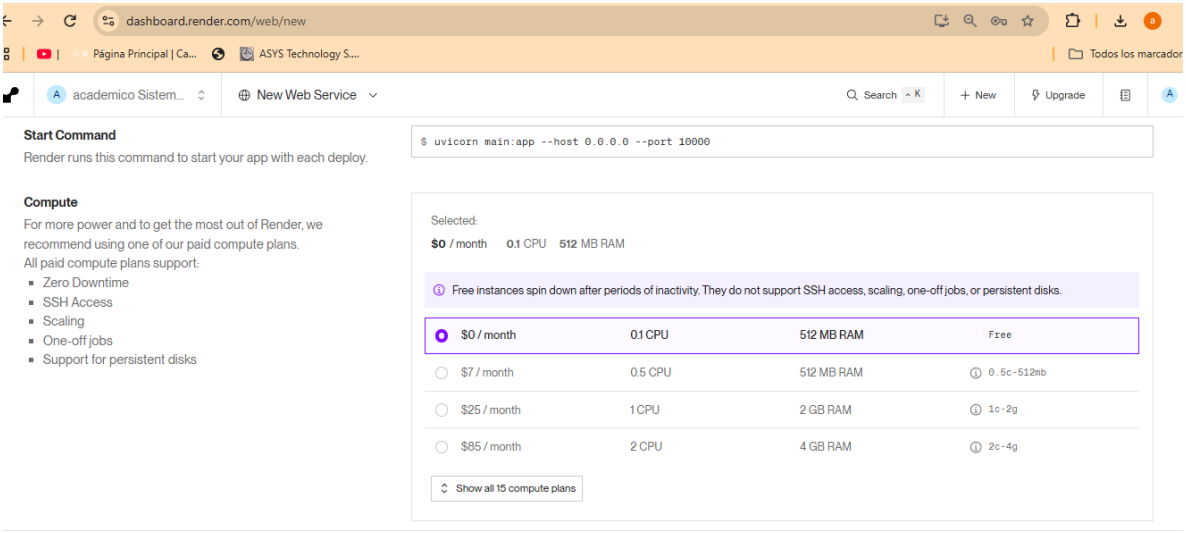
Build Command: el comando que instala las dependencias, normalmente `pip install -r requirements.txt`.

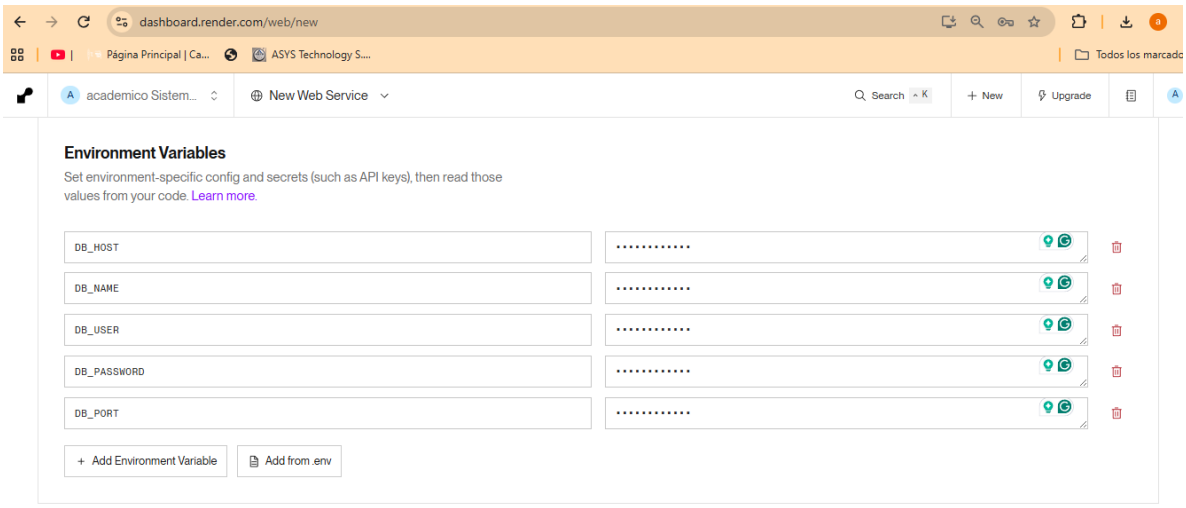
*Start Command**: el comando que enciende la API, normalmente algo como `uvicorn main:app --host 0.0.0.0 --port 10000` (ajustando `main:app` al nombre real del archivo y la variable de tu aplicación FastAPI).

Environment Variables (variables de entorno): aquí es donde se debe agregar, por ejemplo, la cadena de conexión a la base de datos de Neon (`DATABASE_URL`), en lugar de escribirla directamente en el código. Esto protege la información sensible del proyecto.

Nombre: api_estudiante



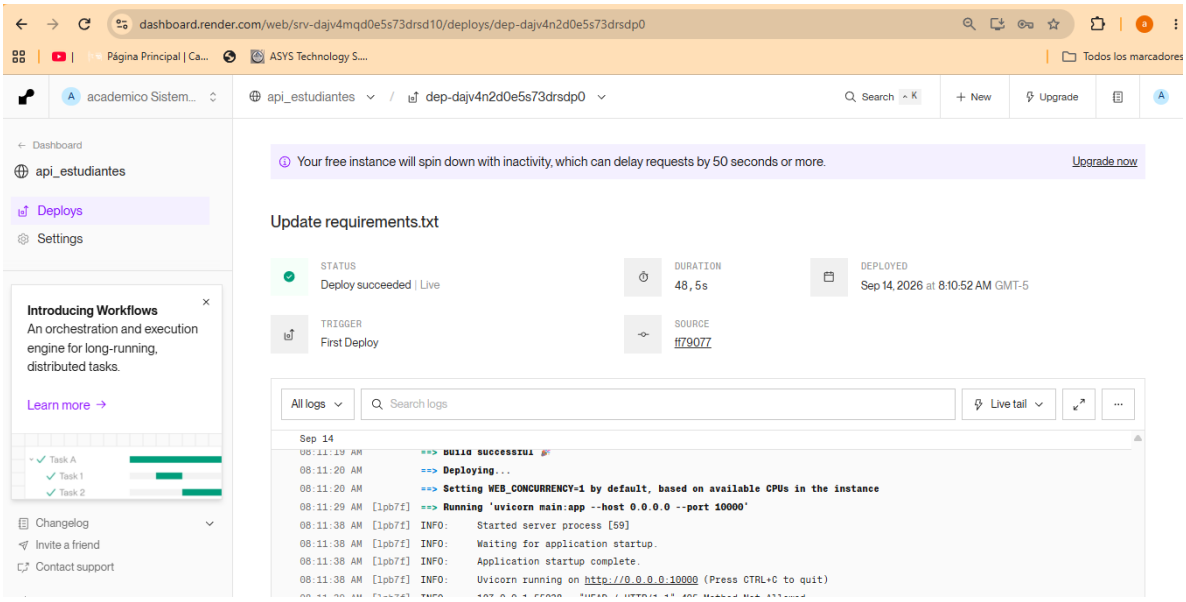




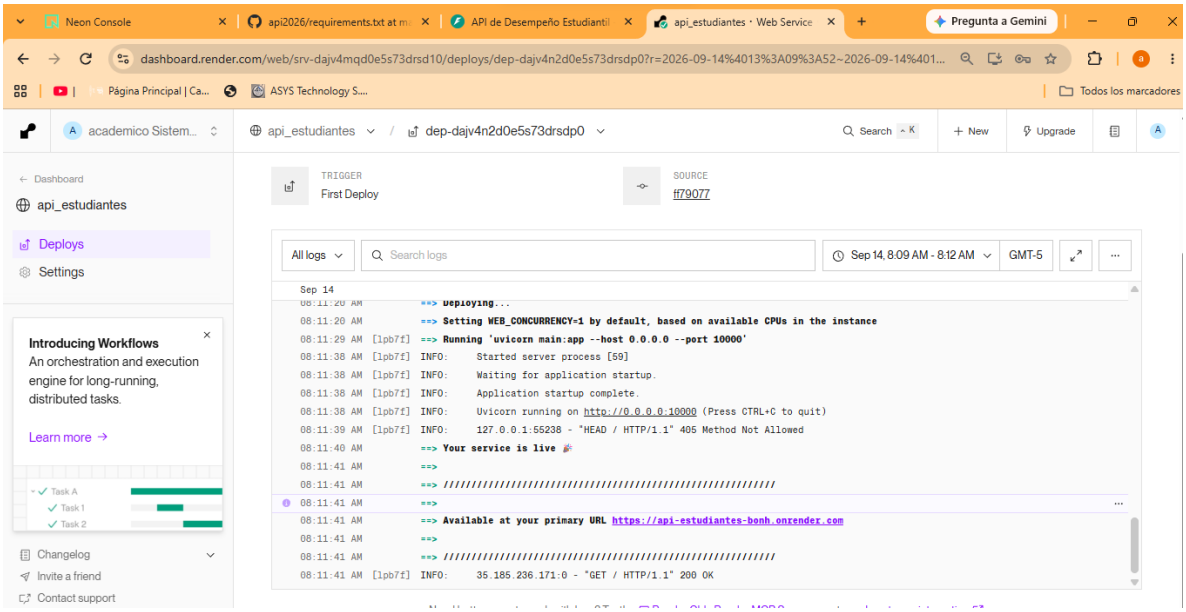
Clic en deploy

Al hacer clic en **Deploy**, Render ejecuta automáticamente el *Build Command* y luego el *Start Command*. Este proceso se puede seguir en tiempo real en la pestaña de **Logs**, donde aparecen los mensajes de instalación de librerías y, si algo falla, el mensaje de error correspondiente. Revisar los logs es el primer paso para solucionar cualquier problema durante el despliegue.

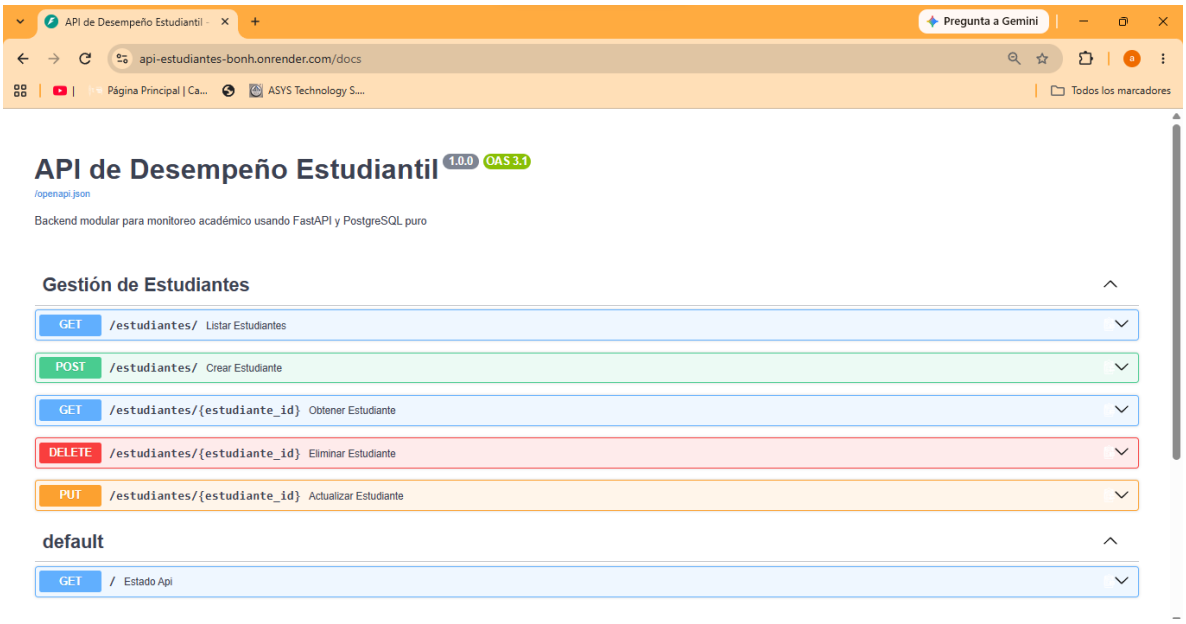
Una vez el despliegue termina exitosamente, Render entrega una **URL pública** (con el dominio `onrender.com`) desde la cual la API queda disponible para cualquier persona en internet. Al agregar `/docs` al final de la URL, FastAPI muestra automáticamente una interfaz interactiva llamada **Swagger UI**, generada a partir del estándar **OpenAPI**. Desde esta interfaz se pueden probar directamente los endpoints (GET, POST, etc.) sin necesidad de escribir código adicional, lo cual es muy útil para verificar que la API responde correctamente antes de conectarla con otra aplicación



Validar la URL



https://api-estudiantes-bonh.onrender.com/docs



Probar las rutas de la Api

